Experiment 7 – Apply Adversarial Search on given problem

Learning Objective: To understand and implement the Minimax algorithm for adversarial search problems, and to analyse optimal decision-making strategies in competitive, multi-agent environments using Python.

Tools: Visual Code Studio

Theory: -

In Artificial Intelligence, adversarial search algorithms are used in environments where multiple agents interact with conflicting goals. These problems are commonly found in games such as chess, tic-tac-toe, and checkers, where one player's gain is another player's loss. Such environments are referred to as competitive or zero-sum environments. Adversarial search helps in determining the optimal strategy for a player by considering the possible moves of the opponent.

This experiment focuses on the Minimax algorithm, which is a fundamental technique used for decision-making in two-player turn-based games.

1. Game Representation in Adversarial Search
Adversarial problems are represented using a game tree. In this tree:

- Each node represents a state of the game.
- The root node represents the current state.
- Edges represent possible moves.
- Leaf nodes represent terminal states (win, lose, or draw).

The players are typically referred to as:

- MAX (Maximizer): tries to maximize the score
- MIN (Minimizer): tries to minimize the score

2. Minimax Algorithm
The Minimax algorithm is used to determine the optimal move for a player by assuming that the opponent also plays optimally. It explores all possible game states up to a certain depth and evaluates the outcome of each move.

The working of the Minimax algorithm is as follows:

- The algorithm starts from the root node and recursively explores all possible moves.
- At MAX nodes, it selects the move with the maximum value.
- At MIN nodes, it selects the move with the minimum value.
- This process continues until it reaches terminal nodes, which are assigned utility values (e.g., +1 for win, -1 for loss, 0 for draw).
- The values are then propagated back up the tree to determine the best move at the root.

3. Working Principle
Minimax operates on a depth-first search basis:

- It generates the full game tree (or up to a limited depth).
- Evaluates leaf nodes using a utility function.

- Backtracks to compute optimal values for intermediate nodes.

This ensures that the decision made is optimal, assuming both players play rationally.

4. Properties of Minimax Algorithm

- Complete: Yes, if the game tree is finite.
- Optimal: Yes, it guarantees the best possible move against an optimal opponent.
- Time Complexity: Exponential, typically O(b^d), where b is branching factor and d is depth.
- Space Complexity: Also depends on depth due to recursive exploration.

5. Limitations of Minimax

- Computationally expensive for large game trees.
- Requires exploring many nodes, even those that may not affect the final decision.
- Performance decreases as the depth of the tree increases.

6. Applications of Adversarial Search
The Minimax algorithm is widely used in:

- Game AI (tic-tac-toe, chess, checkers)
- Decision-making systems in competitive environments
- Strategy planning problems

Overall, adversarial search using the Minimax algorithm provides a structured approach to decision-making in competitive scenarios. It forms the foundation for more advanced techniques such as Alpha-Beta Pruning, which improves efficiency by reducing unnecessary node exploration.

<u>Code & Output</u>: -

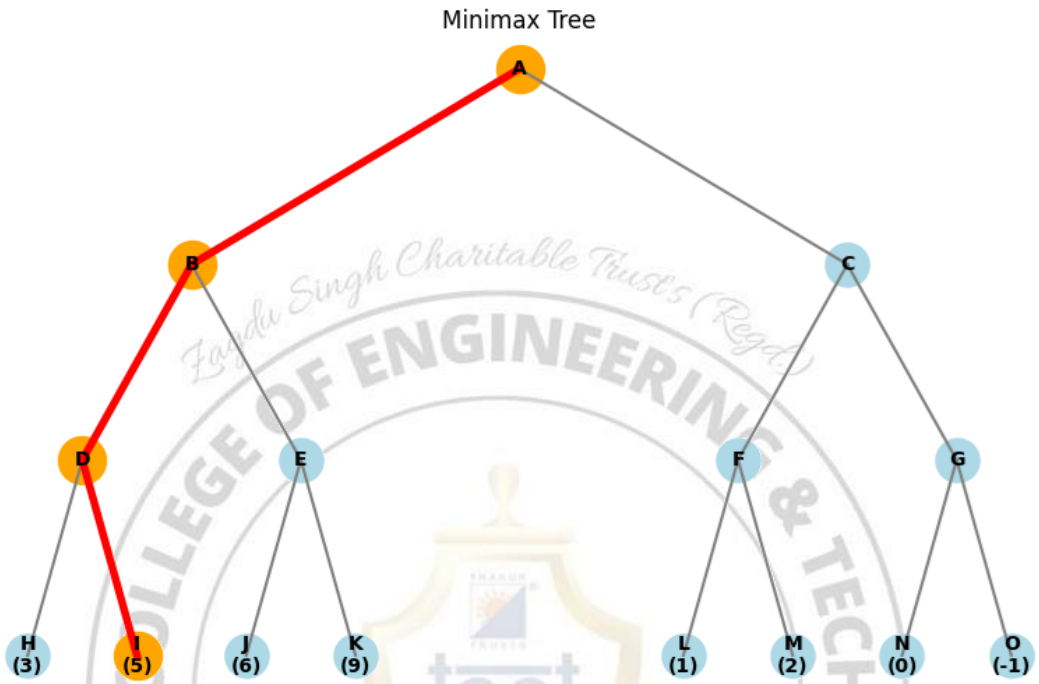
Input: -

```
import matplotlib.pyplot as plt

# -------- TREE STRUCTURE --------
tree = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': ['H', 'I'],
    'E': ['J', 'K'],
    'F': ['L', 'M'],
    'G': ['N', 'O'],
    'H': [], 'I': [], 'J': [], 'K': [],
    'L': [], 'M': [], 'N': [], 'O': []
}

# -------- LEAF VALUES --------
values = {
    'H': 3, 'I': 5,
    'J': 6, 'K': 9,
    'L': 1, 'M': 2,
    'N': 0, 'O': -1
}

# -------- NODE POSITIONS --------
pos = {
    'A': (0, 5),

    'B': (-3, 4), 'C': (3, 4),

    'D': (-4, 3), 'E': (-2, 3),
    'F': (2, 3), 'G': (4, 3),

    'H': (-4.5, 2), 'I': (-3.5, 2),
    'J': (-2.5, 2), 'K': (-1.5, 2),
```

```
    'L': (1.5, 2), 'M': (2.5, 2),
    'N': (3.5, 2), 'O': (4.5, 2)
}

# -------- MINIMAX FUNCTION --------
def minimax(node, depth, is_max):

    # Leaf node
    if node not in tree or len(tree[node]) == 0:
        return values[node], [node]

    paths = []

    if is_max:
        best_val = float('-inf')
        best_path = []

        for child in tree[node]:
            val, path = minimax(child, depth + 1,
False)
            if val > best_val:
                best_val = val
                best_path = [node] + path

        return best_val, best_path

    else:
        best_val = float('inf')
        best_path = []

        for child in tree[node]:
            val, path = minimax(child, depth + 1,
True)
            if val < best_val:
                best_val = val
                best_path = [node] + path

        return best_val, best_path


# -------- DRAW FUNCTION --------
def draw_tree(ax, title, path=None):

    # Draw edges
    for u in tree:
        for v in tree[u]:
            x1, y1 = pos[u]
            x2, y2 = pos[v]
            ax.plot([x1, x2], [y1, y2], color='gray')

    # Highlight optimal path
    if path:
        for i in range(len(path) - 1):
            x1, y1 = pos[path[i]]
            x2, y2 = pos[path[i+1]]
            ax.plot([x1, x2], [y1, y2], linewidth=4,
color='red')

    # Draw nodes
    for node, (x, y) in pos.items():
        if path and node in path:
            ax.scatter(x, y, s=600, color='orange')
        else:
            ax.scatter(x, y, s=500, color='lightblue')

        label = node
        if node in values:
            label += f"\n({values[node]})"

        ax.text(x, y, label, ha='center', va='center',
fontweight='bold')

    ax.set_title(title)
    ax.axis('off')


# -------- MAIN --------
optimal_value, optimal_path = minimax('A', 0,
True)

print("Optimal Value:", optimal_value)
print("Optimal Path:", optimal_path)

# -------- PLOT --------
fig, ax = plt.subplots(figsize=(10, 6))
draw_tree(ax, "Minimax Tree", optimal_path)
plt.show()
```

Output: -

Optimal Value: 5
Optimal Path: ['A', 'B', 'D', 'I']



Fakir Singh Charitable Trust's (Regd.)
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY
Estd. 2001
ISO 9001 : 2015 Certified
NBA and NAAC Accredited

Learning Outcomes: The student should have the ability to:

LO 7.1: Apply the Minimax algorithm to solve adversarial search problems in two-player game environments.
LO 7.2: Analyse game trees and evaluate optimal moves using MAX and MIN strategies
LO 7.3: Understand decision-making in competitive environments assuming rational opponents.
LO 7.4: Implement adversarial search techniques for game-based problem-solving using Python.

Course Outcomes:

Upon completion of this experiment, students will be able to understand and apply adversarial search techniques using the Minimax algorithm to make optimal decisions in competitive environments, and analyse strategic interactions in Artificial Intelligence applications.

Conclusion:

Jagdu Singh Charitable Trust's (Regd.)
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY
Estd. 2001
ISO 9001 : 2015 Certified
NBA and NAAC Accredited

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				